



Fundamentals of Database System

Chapter 7: Relational Model & Relational Algebra

Compiled by: Diriba G. (MSc. in CSE)

- ❖ Introduction to;
 - ❖ Relational model and Relational algebra
 - ❖ Relational algebra operations
 - ❖ Selection
 - ❖ Projection
 - ❖ Cartesian product
 - ❖ Union and Set difference
- ❖ Different relational Algebra Operations from Set theory and additional operation with example

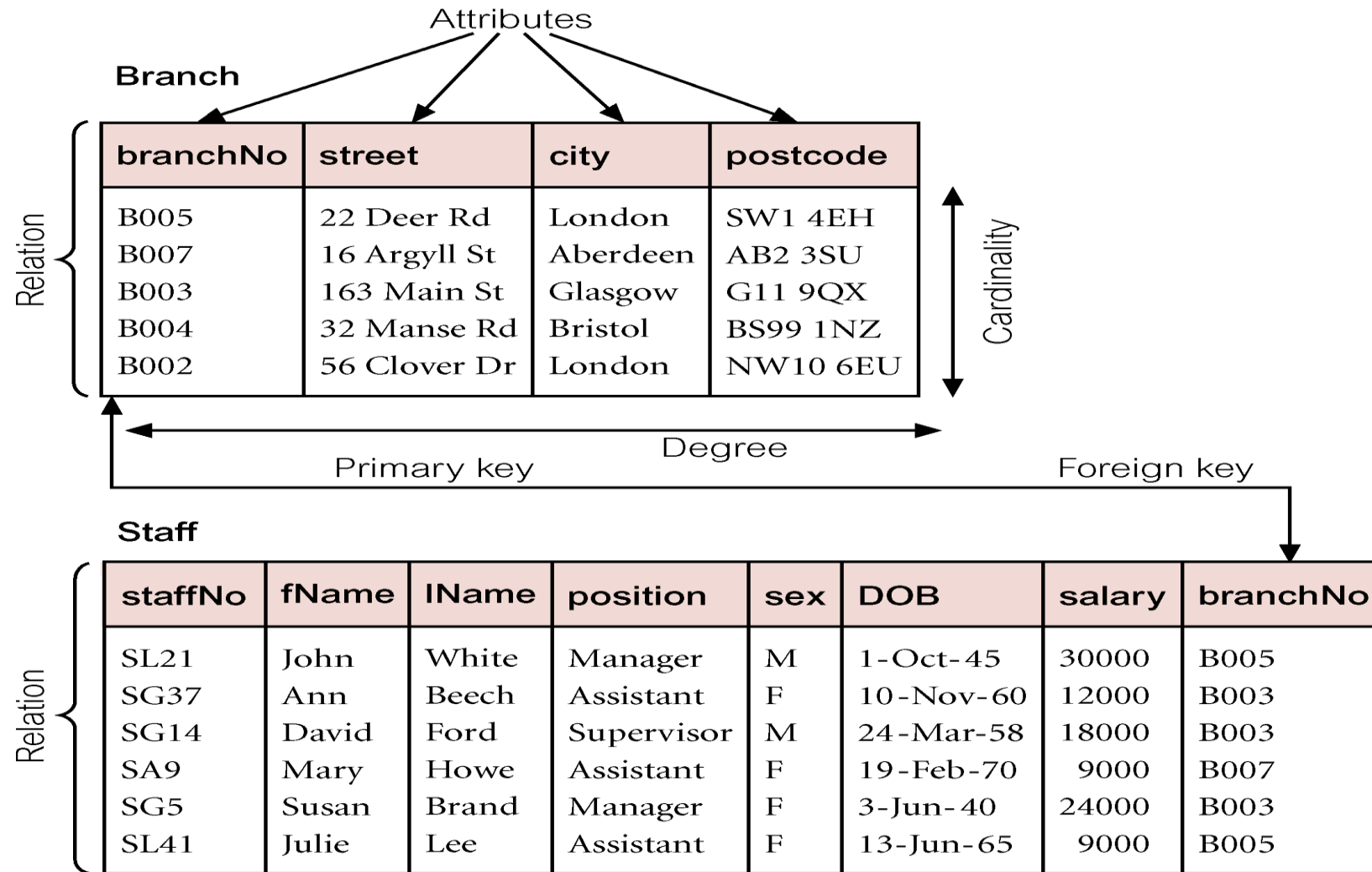
What is Relational Model refers to?

- ❖ Relational model supports simple, powerful query languages in which allows manipulation and retrieval of data from a database.

Relational Model Terminology

- ❖ **Tuple** is a row of a relation.
- ❖ **Degree** is the number of attributes in a relation.
- ❖ **Cardinality** is the number of tuples in a relation.
- ❖ **Relational Database** is a collection of normalized relations with distinct relation names.

Instances of Branch and Staff (part) Relations



Database Relations

- ❖ Relation schema

- ❖ Named relation defined by a set of attribute and domain name pairs.

- ❖ Relational database schema

- ❖ Set of relation schemas, each with a distinct name.

Properties of Relations

- ❖ Relation name is distinct from all other relation names in relational schema.
- ❖ Each cell of relation contains exactly one atomic (single) value.
- ❖ Each attribute has a distinct name.
- ❖ Values of an attribute are all from the same domain.
- ❖ Each tuple is distinct; there are no duplicate tuples.
- ❖ Order of attributes has no significance.
- ❖ Order of tuples has no significance, theoretically.

Relational Algebra

- ❖ Relational algebra is a formal language associated with the relational model.
- ❖ we may describe the relational algebra as procedural language in which user tells the system exactly **what** and **how** to manipulate the data. And also it can be used to tell the DBMS how to build a new relation from one or more relations in the database.

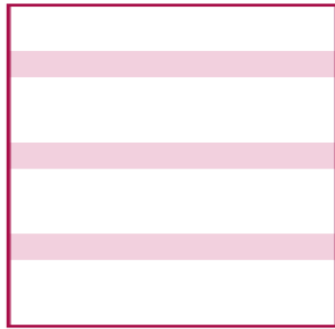
Relational algebra operations

- ❖ The basic set of operations for the relational model is known as the relational algebra. These operations enable a user to specify basic retrieval requests.
- ❖ There are different basic operations that could be applied on relations based on the requirement. However these five operation listed below are the most important and basic operations in relational algebra:
- ❖ These operations are:

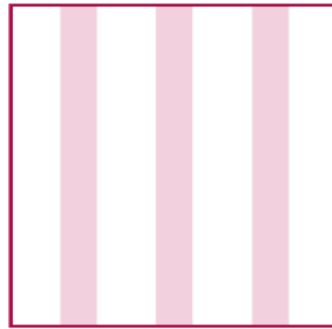
Cont....

- ❖ **Selection.** It selects a subset of rows from a relation.
- ❖ **Projection.** Deletes unwanted columns from a relation
- ❖ **Cartesian product.** Allows to concatenate a tuple from one relation with all the tuples from the other relation.
- ❖ **Union.** Tuples in relation R1, or in relation R2.
- ❖ **Set Difference.** Tuples in relation R1, but not in relation R2
- ❖ These operations perform most of the data retrieval needed. And also there is Join, Intersection which can be expressed in terms of this 5 basic operations.

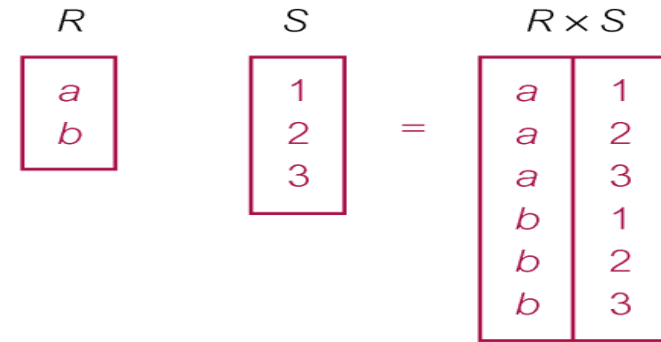
Cont..



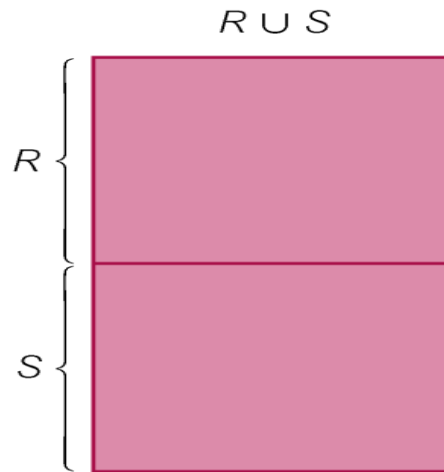
(a) Selection



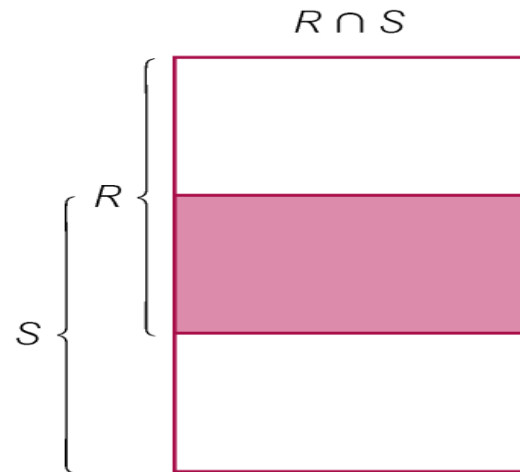
(b) Projection



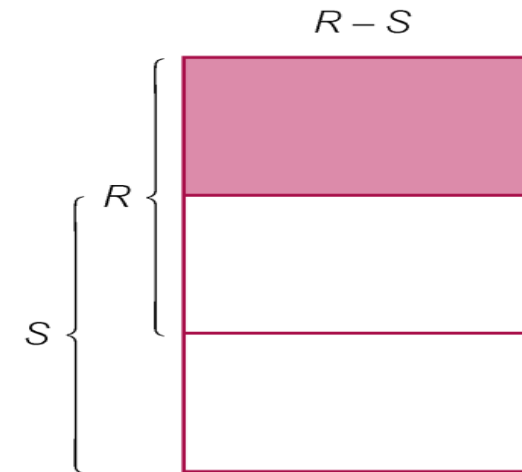
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

Relational Algebra Operations from Set Theory: SET DIFFERENCE (cont.)

- ❖ SET DIFFERENCE (also called MINUS or EXCEPT) is denoted by $R - S$
- ❖ The result of $R - S$, is a relation that includes all tuples that are in R but not in S
 - ❖ The attribute names in the result will be the same as the attribute names in R
- ❖ The two operand relations R and S must be “type compatible”

Some properties of UNION, INTERSECT, and SET DIFFERENCE

- ❖ Notice that both union and intersection are *commutative* operations; that is
 - ❖ $R \cup S = S \cup R$, and $R \cap S = S \cap R$
- ❖ Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are *associative* operations; that is
 - ❖ $R \cup (S \cup T) = (R \cup S) \cup T$
 - ❖ $(R \cap S) \cap T = R \cap (S \cap T)$
- ❖ The minus operation is not commutative; that is, in general
 - ❖ $R - S \neq S - R$

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT

❖ CARTESIAN (or CROSS) PRODUCT Operation

- ❖ This operation is used to combine tuples from two relations in a combinatorial fashion.
- ❖ Denoted by $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$
- ❖ Result is a relation Q with degree $n + m$ attributes:
 - ❖ $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- ❖ The resulting relation state has one tuple for each combination of tuples—one from R and one from S .

Cont..

- ❖ Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R * n_S$ tuples.
- ❖ The two operands do NOT have to be "type compatible"

Binary Relational Operations: JOIN

- ❖ JOIN Operation (denoted by $\bowtie$)
 - ❖ The sequence of CARTESIAN PRODECT followed by SELECT is used quite commonly to identify and select related tuples from two relations
 - ❖ A special operation, called JOIN combines this sequence into a single operation

Cont..

- ❖ This operation is very important for any relational database with more than a single relation, because it allows us *combine related tuples* from various relations
- ❖ The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is $R \text{ }_{\langle \text{join condition} \rangle} S$
- ❖ where R and S can be any relations that result from general *relational algebra expressions*.

Some properties of JOIN

- ❖ Consider the following JOIN operation:
- ❖ $R(A_1, A_2, \dots, A_n) \bowtie S(B_1, B_2, \dots, B_m)$
 $R.A_i = S.B_j$
- ❖ Result is a relation Q with degree $n + m$ attributes:
 - ❖ $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- ❖ The resulting relation state has one tuple for each combination of tuples— r from R and s from S , but *only if they satisfy the join condition* $r[A_i] = s[B_j]$
- ❖ Hence, if R has n_R tuples, and S has n_S tuples, then the join result will generally have *less than* $n_R * n_S$ tuples.
- ❖ Only related tuples (based on the join condition) will appear in the result

Cont...

- ❖ The general case of JOIN operation is called a Theta-join: $R \bowtie_{\theta} S$
- ❖ The join condition is called *theta*
- ❖ *Theta* can be any general boolean expression on the attributes of R and S; for example:
 - ❖ $R.A_i < S.B_j \text{ AND } (R.A_k = S.B_l \text{ OR } R.A_p < S.B_q)$
- ❖ Most join conditions involve one or more equality conditions “AND”ed together; for example:
 - ❖ $R.A_i = S.B_j \text{ AND } R.A_k = S.B_l \text{ AND } R.A_p = S.B_q$

Binary Relational Operations: EQUIJOIN

❖EQUIJOIN Operation

- ❖The most common use of join involves join conditions with *equality comparisons* only. Such a join, where the only comparison operator used is $=$, is called an EQUIJOIN.
- ❖In the result of an EQUIJOIN we always have one or more pairs of attributes (whose names need not be identical) that have identical values in every tuple.

- ❖ NATURAL JOIN Operation is another variation of JOIN called NATURAL JOIN was created to get rid of the second (superfluous) attribute in an EQUIJOIN condition.
 - ❖ because one of each pair of attributes with identical values is superfluous
- ❖ The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, *have the same name* in both relations
- ❖ If this is not the case, a renaming operation is applied first.

Complete Set of Relational Operations

❖ The set of operations including SELECT σ , PROJECT π , UNION $\cup$, DIFFERENCE $-$, RENAME ρ , and CARTESIAN PRODUCT $\times$ is called a *complete set* because any other relational algebra expression can be expressed by a combination of these five operations.

❖ For example:

$$\text{❖ } R \cap S = (R \cup S) - ((R - S) \cup (S - R))$$

$$\text{❖ } R \text{ <join condition> } S = \sigma \text{ <join condition> } (R \times S)$$

Recap of Relational Algebra Operations

Table 6.1
Operations of Relational Algebra

Operation	Purpose	Notation
SELECT	Selects all tuples that satisfy the selection condition from a relation R .	$\sigma_{\langle \text{selection condition} \rangle}(R)$
PROJECT	Produces a new relation with only some of the attributes of R , and removes duplicate tuples.	$\pi_{\langle \text{attribute list} \rangle}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \bowtie_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{\langle \text{join condition} \rangle} R_2$, OR $R_1 *_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$ OR $R_1 * R_2$
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2 ; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2 .	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Additional Relational Operations (cont.)

- ❖ As we have seen in previous section there are five basic operation of relational algebra. In addition to these different types of join operations were exist. For instance in natural join and equijoin, tuples without a *matching* (or *related*) tuple are eliminated from the join result
 - ❖ Tuples with null in the join attributes are also eliminated
 - ❖ This amounts to loss of information.

- ❖ A set of operations, called OUTER joins, can be used when we want to keep all the tuples in R, or all those in S, or all those in both relations in the result of the join, regardless of whether or not they have matching tuples in the other relation.
- ❖ The left outer join operation keeps every tuple in the first or left relation R in R $\bowtie$ S; if no matching tuple is found in S, then the attributes of S in the join result are filled or “padded” with null values.

Cont...

- ❖ A similar operation, right outer join, keeps every tuple in the second or right relation S in the result of $R \bowtie S$.
- ❖ A third operation, full outer join, that keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

❖ OUTER UNION Operations

- ❖ The outer union operation was developed to take the union of tuples from two relations if the relations are *not type compatible*.
- ❖ This operation will take the union of tuples in two relations $R(X, Y)$ and $S(X, Z)$ that are **partially compatible**, meaning that only some of their attributes, say X , are type compatible.
- ❖ The attributes that are type compatible are represented only once in the result, and those attributes that are not type compatible from either relation are also kept in the result relation $T(X, Y, Z)$.

Cont....

❖ Example Consider the following relations to understand additional join operation in relational algebra. The first one is student relation which has the following properties with certain records and the second is course relation which has also different properties.

Student relation

sno	sname	sage
S1	n1	21
S2	n2	22
S3	n3	23
S4	n4	24

Course relation

sno	cname	fees
S1	C1	1000
S3	C3	3000
S5	C5	5000

Cont....

A). Natural join operation.

S no	sname	sage	cname	fees
S1	n1	21	C1	1000
S3	n3	23	C3	3000

Student $\bowtie$ course
example of natural join

B). Left –outer Join Operation

S no	sname	sage	cname	fees
S1	n1	21	C1	1000
S2	n2	22	null	null
S3	n3	23	C3	3000
S4	n4	24	null	null

Student $\Join$ course
example of Left-outer join

Cont....

A). Right outer join operation.

sno	sname	sage	cname	fees
S1	n1	21	C1	1000
S3	n3	23	C3	3000
S5	null	null	C5	5000

Student $\bowtie$ course
example of right-outer join

B). Full outer join Operation

sno	sname	sage	cname	fees
S1	n1	21	C1	1000
S2	n2	22	null	null
S3	n3	23	C3	3000
S4	n4	24	null	null
S5	null	null	C5	5000

Student $\bowtie$ course
example of full outer join.

Thank You!